

Fondements informatiques I

Cours 4: Fonctions

Sorina Ionica `sorina.ionica@uvsq.fr`

Bilal FAYE `bilal.faye@uvsq.fr`

Les fonctions

Pourquoi utiliser une fonction ?

Une fonction permet de **regrouper plusieurs instructions** pour réaliser une tâche précise.

Elle peut ensuite être **réutilisée** plusieurs fois dans un programme.

Idée générale

Données $\longrightarrow$ **Fonction** $\longrightarrow$ **Résultat**

Exemple

Une fonction pourrait prendre deux nombres en entrée et calculer leur somme.

3, 5 $\longrightarrow$ calcul de la somme $\longrightarrow$ 8

Comment fonctionne une fonction ?

Une fonction peut recevoir des données et produire un résultat

Une fonction est généralement composée de trois éléments :

Entrées

Les données
nécessaires

Traitement

Les instructions
à exécuter

Sortie

Le résultat
obtenu

Exemple

Deux nombres $\longrightarrow$ **addition** $\longrightarrow$ leur somme

Les fonctions en mathématiques

Une fonction

En mathématiques, une fonction associe une ou plusieurs **entrées** à un **résultat**.

Notation

$$f : x \rightarrow f(x)$$

f : nom de la fonction

x : argument

$f(x)$: résultat

Exemples

$$f(x) = ax + b$$

$$f(x, y) = x + y$$

Autres fonctions

$$f(x) = x^2$$

$$f(x) = 2^x$$

$$f(x) = |x|$$

Définir une fonction en Python

Syntaxe

```
def nom_fonction(arg1, arg2, ...):  
    instruction 1  
    instruction 2  
    ...  
    insstruction n
```

- `def` indique que l'on définit une fonction.
- On indique ensuite le **nom** de la fonction et ses **arguments** entre parenthèses.
- Le **corps de la fonction** est un bloc d'instructions **indenté**.

Lien avec les mathématiques

$$f(x) \longleftrightarrow \text{nom_fonction(argument)}$$

Définir une fonction en Python

Exemple

```
def multiplie(x):  
    """Multiplie x par 2 et affiche le résultat."""  
    print(x * 2)  
multiplie(7)  
multiplie("aaa")
```

- `multiplie` est le **nom** de la fonction.
- `x` est son **argument**.
- `multiplie(7)` appelle la fonction avec `x = 7`.

Docstring

La chaîne entre triples guillemets `""" ... """` décrit le rôle de la fonction. Elle peut notamment être consultée avec `help()`.

L'instruction return

Rôle de return

return permet à une fonction de **renvoyer une valeur** au programme qui l'a appelée.

Une fonction peut ne pas avoir de return : dans ce cas, elle renvoie None.

Avec return

```
def multiplie(x):  
    return x * 2  
print(type(multiplie(2)))
```

La fonction renvoie 4.

Sans return

```
def multiplie(x):  
    print(x * 2)  
print(type(multiplie(2)))
```

La fonction renvoie None.

À retenir

print() affiche une valeur

vs.

return renvoie une valeur

Signature d'une fonction

Définition

La **signature d'une fonction** décrit les éléments qui permettent de caractériser la fonction : **son nom, ses paramètres et leurs types**, ainsi que le **type de la valeur renvoyée**.

En Python

Les types des paramètres et le type de retour **ne sont pas obligatoires** dans la définition.

Python détermine les types lors de l'exécution.

Que se passe-t-il lors de l'appel ?

```
def somme(x, y):  
    return x + y  
print(somme("hello", 1))
```

Retourner plusieurs valeurs

Une fonction peut renvoyer plusieurs valeurs

En Python, une instruction `return` peut renvoyer plusieurs valeurs séparées par des virgules.

Ces valeurs sont regroupées dans un **tuple**.

Exemple

```
def ordonne(a, b):  
    if a < b:  
        return a, b  
    else:  
        return b, a  
print(ordonne(10, 5))
```

Résultat

(5, 10)

La fonction renvoie donc un **tuple contenant deux valeurs**.

Sortir de la fonction

Dès qu'une fonction atteint une instruction `return`, elle **s'arrête immédiatement**. Les instructions placées après ce `return` ne sont donc pas exécutées.

Exemple

```
def un():  
    return 1  
    print("Je ne serai jamais atteint")  
print(un())
```

Dans une boucle

```
def cherche(x, l):  
    for elem in l:  
        if x == elem:  
            return 1  
    return 0  
print(cherche(10, [1, 10, 0])) # 1
```

Dès que 10 est trouvé, `return 1` termine la fonction : les éléments suivants ne sont pas parcourus.

Les fonctions existantes en Python

Pas besoin de tout programmer soi-même

Python fournit de nombreuses **fonctions déjà définies**. On peut les appeler directement dans notre programme.

Quelques fonctions que vous connaissez déjà

```
print("Bonjour")      # affiche une valeur
type(3.14)             # donne le type de la valeur
len("Bonjour")        # donne la longueur
```

À retenir

Une fonction existante s'utilise comme une fonction que l'on a définie nous-mêmes : **nom(arguments)**.

Les fonctions d'un module

Certaines fonctions sont regroupées dans des modules

Un **module** est un ensemble de fonctions et d'outils déjà programmés que l'on peut utiliser.

Pour utiliser un module, on commence par l'**importer**.

Exemple avec le module `math`

```
import math  
print(math.sqrt(25))
```

Comment lire `math.sqrt(25)` ?

`math` : le module `sqrt` : la fonction

`math.sqrt(25) = 5.0`

Que fait ce programme ?

```
import math
def calcule(x):
    if x > 0:
        return math.sqrt(x)
    else:
        return 0

i = 1
while i <= 5:
    for x in [i, i + 1]:
        print(calcule(x))
    i = i + 1
```